

Big Data – Assignment 2: Spark Pipeline

By Jia Hao Huang Xia

GitHub Repository: https://github.com/JiaHaoHuangXia/Assignment_2.git

1. Spark RDDs (15%)

In this part you will work with RDDs to appreciate what they can do and what makes them painful for structured data. This connects back to Session 05.

1.1 Load as RDD

Load one month (January 2024) as a Parquet file using the DataFrame API, then convert it to an RDD with `.rdd``. Print the first 3 elements.

What type is each element?

```
[Stage 2:=====] (2 + 2) / 4]
Row(VendorID=2, tpep_pickup_datetime=datetime.datetime(2024, 1, 1, 0, 57, 55), tpep_dropoff_datetime=datetime.datetime(2024, 1, 1, 17, 43), passenger_count=1, trip_distance=1.72, RatecodeID=1, store_and_fwd_flag='N', PULocationID=186, DOLocationID=79, payment_type=2, fare_amount=17.7, extra=1.0, mta_tax=0.5, tip_amount=0.0, tolls_amount=0.0, improvement_surcharge=1.0, total_amount=22.7, congestion_surcharge=2.5, Airport_fee=0.0)
<class 'pyspark.sql.types.Row'>

Row(VendorID=1, tpep_pickup_datetime=datetime.datetime(2024, 1, 1, 0, 3), tpep_dropoff_datetime=datetime.datetime(2024, 1, 1, 0, 9, 36), passenger_count=1, trip_distance=1.8, RatecodeID=1, store_and_fwd_flag='N', PULocationID=140, DOLocationID=236, payment_type=1, fare_amount=10.0, extra=3.5, mta_tax=0.5, tip_amount=3.75, tolls_amount=0.0, improvement_surcharge=1.0, total_amount=18.75, congestion_surcharge=2.5, Airport_fee=0.0)
<class 'pyspark.sql.types.Row'>

Row(VendorID=1, tpep_pickup_datetime=datetime.datetime(2024, 1, 1, 0, 17, 6), tpep_dropoff_datetime=datetime.datetime(2024, 1, 1, 0, 35, 1), passenger_count=1, trip_distance=4.7, RatecodeID=1, store_and_fwd_flag='N', PULocationID=236, DOLocationID=79, payment_type=1, fare_amount=23.3, extra=3.5, mta_tax=0.5, tip_amount=3.0, tolls_amount=0.0, improvement_surcharge=1.0, total_amount=31.3, congestion_surcharge=2.5, Airport_fee=0.0)
<class 'pyspark.sql.types.Row'>
```

Each element in the RDD is a `pyspark.sql.types.Row` object. Each row contains the values for one taxi trip record.

1.2 RDD Transformations

Using the RDD from 1.1:

1. Count the total number of trips.

```
print("Total_trip: ", rdd_jan.count())
```

```
[Stage 13:=====]
Total_trip: 2964624
```

2. Filter for trips with ``passenger_count > 4``. How many are there?

```
passenger = rdd_jan.filter(lambda row: row.passenger_count is not None and row.passenger_count > 4)
print("Trips with passenger_count > 4:", passenger.count())
```

```
[Stage 14:=====] (6 + 2) / 8]
Trips with passenger_count > 4: 55919
```

3. Compute the average ``trip_distance`` using ``map`` and ``reduce`` (not DataFrames)

```
trip_distance = rdd_jan.map(lambda row: (row.trip_distance, 1)).reduce(lambda a, b: (a[0] + b[0], a[1] + b[1]))
avg_trip_distance = trip_distance[0] / trip_distance[1]
print("Average trip distance:", avg_trip_distance)
```

```
[Stage 5:=====> (6 + 2) / 8]
Average trip distance: 3.6521691789583146
```

1.3 WordCount-style aggregation

Using RDD operations only (`map`, `reduceByKey`):

1. Compute total revenue (`total\_amount`) per `PULocationID` (pickup location).
2. Find the top 5 pickup locations by revenue.

```
revenue_per_location = rdd_jan.filter(lambda row: row.PULocationID is not None and row.total_amount is not None)\
    .map(lambda row: (row.PULocationID, row.total_amount)).reduceByKey(lambda a, b: a + b)

top_5 = (revenue_per_location.sortBy(lambda x: x[1], ascending=False).take(5))

for location in top_5:
    print(location)
```

```
(132, 11121928.529997837)
(138, 5820969.9599999)
(161, 3369045.009999964)
(230, 2793051.6799999666)
(237, 2776195.3199999486)
```

1.4 Reflection

In your report, answer: Compare your RDD code from 1.2-1.3 with the equivalent DataFrame code you'll write in Part 2. Which is more readable? Which required more manual work? Could Spark optimize your RDD code automatically?

The DataFrame code was more readable because it used high-level operations such as filter, groupBy, and agg with column names directly. The RDD code required much more manual work by needing to use map, filter, reduce, and tuple transformations. It also required manual aggregation logic, while DF simplified these operations significantly.

Spark could not optimize the RDD code as effectively because RDDs do not contain schema information. On the other hand, DFs allow optimizations like column pruning, filter pushdown, and optimized execution plans automatically.

2. Dataframes: Cleaning and Transforming (25%)

Now switch to DataFrames for the full year of data. This connects back to Session 06.

2.1 Load and combine

Read all 12 monthly Parquet files into a single DataFrame. Print the schema and total row

count.

Hint: `spark.read.parquet()` accepts a glob pattern like ``/app/data/yellow_tripdata_2024-*.parquet``.

Using the same spark configuration as session 6:

```
df = spark.read.parquet("/app/data/yellow_tripdata_2024-*.parquet")
df.printSchema()
```

```
root
|-- VendorID: integer (nullable = true)
|-- tpep_pickup_datetime: timestamp_ntz (nullable = true)
|-- tpep_dropoff_datetime: timestamp_ntz (nullable = true)
|-- passenger_count: long (nullable = true)
|-- trip_distance: double (nullable = true)
|-- RatecodeID: long (nullable = true)
|-- store_and_fwd_flag: string (nullable = true)
|-- PULocationID: integer (nullable = true)
|-- DOLocationID: integer (nullable = true)
|-- payment_type: long (nullable = true)
|-- fare_amount: double (nullable = true)
|-- extra: double (nullable = true)
|-- mta_tax: double (nullable = true)
|-- tip_amount: double (nullable = true)
|-- tolls_amount: double (nullable = true)
|-- improvement_surcharge: double (nullable = true)
|-- total_amount: double (nullable = true)
|-- congestion_surcharge: double (nullable = true)
|-- Airport_fee: double (nullable = true)
```

2.2 Data exploration

Answer these questions using the DataFrame API:

1. How many rows and columns are there?

```
total_rows = df.count()

print("Total rows:", df.count())
print("Total columns:", len(df.columns))
```

```
Total rows: 41169720
Total columns: 19
```

2. Are there any null values? Which columns have them and how many?

Columns with null values

```
null_counts = df.select([count(when(col(c).isNull(), c)).alias(c) for c in df.columns])
null_counts.show(vertical=True, truncate=False)
```

```
-RECORD 0-----
VendorID          | 0
tpep_pickup_datetime | 0
tpep_dropoff_datetime | 0
passenger_count    | 4091232
trip_distance      | 0
RatecodeID         | 4091232
store_and_fwd_flag | 4091232
PULocationID       | 0
DOLocationID       | 0
payment_type       | 0
fare_amount        | 0
extra              | 0
mta_tax            | 0
tip_amount         | 0
tolls_amount       | 0
improvement_surcharge | 0
total_amount       | 0
congestion_surcharge | 4091232
airport_fee        | 4091232
```

3. What is the date range covered? (min/max of `tpep\_pickup\_datetime`). Are there any outliers?

The date range covered is:

```
df.select(
    min("tpep_pickup_datetime").alias("min_pickup"),
    max("tpep_pickup_datetime").alias("max_pickup")
).show(truncate=False)
```

```
[Stage 49:=====> (8 + 4) / 12]
+-----+-----+
|min_pickup      |max_pickup      |
+-----+-----+
|2002-12-31 16:46:07|2026-06-26 23:53:12|
+-----+-----+
```

But since the dataset should only contain 2024 data, this shouldn't be possible. Meaning, there are outliers:

```
df.filter(
  (col("tpep_pickup_datetime") < "2024-01-01") |
  (col("tpep_pickup_datetime") >= "2025-01-01")
).select("tpep_pickup_datetime").show()
```

```
+-----+
|tpep_pickup_datetime|
+-----+
| 2009-01-01 00:35:59|
| 2002-12-31 16:46:07|
| 2009-01-01 00:13:29|
| 2009-01-01 00:01:41|
| 2008-12-31 23:04:30|
| 2002-12-31 23:10:22|
| 2002-12-31 23:19:30|
| 2009-01-01 05:36:19|
| 2025-02-09 03:46:10|
| 2025-03-02 11:38:31|
| 2025-03-23 20:42:06|
| 2008-12-31 23:58:01|
| 2025-01-06 13:52:50|
| 2025-01-16 10:16:49|
| 2009-01-01 00:06:17|
| 2008-12-31 23:03:59|
| 2008-12-31 23:05:26|
| 2009-01-01 14:02:23|
| 2008-12-31 23:03:46|
| 2009-01-01 00:38:49|
+-----+
only showing top 20 rows
```

4. What are the distinct values of `payment\_type`? How many trips per type?

6 types of payment_type:

```
df.groupBy("payment_type").count().orderBy(col("count").desc()).show()
```

```
[Stage 58:=====> (8 + 4) / 12]
+-----+-----+
|payment_type|count|
+-----+-----+
|1|30452159|
|2|5540088|
|0|4091232|
|4|794494|
|3|291743|
|5|4|
+-----+-----+
```

2.3 Data cleaning

The real-world taxi data has quality issues. Clean it:

1. Remove rows where `trip\_distance` <= 0 or `total\_amount` <= 0 (invalid trips).

```
# 2.3
df_clean = df.filter((col("trip_distance") > 0) & (col("total_amount") > 0))
step1 = df_clean.count()
removed_step1 = df.count() - step1
print("Initial rows:", df.count())
print("Removed invalid trips:", removed_step1)
```

Initial rows: 41169720
Removed invalid trips: 1336887

2. Remove rows where `passenger\_count` is null or 0.

```
df_clean = df_clean.filter(col("passenger_count").isNull() & (col("passenger_count") > 0))
step2 = df_clean.count()
removed_step2 = step1 - step2
print("Removed invalid passenger counts:", removed_step2)
```

Removed invalid passenger counts: 4200008

3. Remove outliers: trips where `trip\_distance` > 200` miles or `total_amount` > 5000` dollars.

```
df_clean = df_clean.filter((col("trip_distance") <= 200) & (col("total_amount") <= 5000))
step3 = df_clean.count()
removed_step3 = step2 - step3
print("Removed outliers:", removed_step3)
```

[Stage 109:=====> (2 + 8) / 12]
Removed outliers: 268

4. Remove time anomalies: trips where the pickup date is outside 2024.

```
df_clean = df_clean.filter((col("tpep_pickup_datetime") >= "2024-01-01") & (col("tpep_pickup_datetime") < "2025-01-01"))
step4 = df_clean.count()
removed_step4 = step3 - step4
print("Removed time anomalies:", removed_step4)
```

[Stage 112:=====> (8 + 4) / 12]
Removed time anomalies: 55

Report: how many rows were removed in each step? What percentage of the dataset did you clean out?

```

print("Initial rows:", df.count())
print("Invalid trips removed:", removed_step1)
print("Invalid passenger count removed:", removed_step2)
print("Outliers removed:", removed_step3)
print("Time anomalies removed:", removed_step4)
print("Final rows:", step4)
total_removed = df.count() - step4
percentage_removed = (total_removed / df.count()) * 100
print("Total removed:", total_removed)
print(f"Total removed percentage: {percentage_removed:.2f}%")

```

```

Initial rows: 41169720
Invalid trips removed: 1336887
Invalid passenger count removed: 4200008
Outliers removed: 268
Time anomalies removed: 55
Final rows: 35632502
Total removed: 5537218
Total removed percentage: 13.45%

```

2.4 Feature engineering

Add the following columns to your cleaned DataFrame:

1. ``trip_duration_minutes``: computed from pickup and dropoff timestamps.
2. ``hour_of_day``: extracted from ``tpep_pickup_datetime`` (0-23).
3. ``day_of_week``: extracted from ``tpep_pickup_datetime`` (1=Monday, 7=Sunday).
4. ``month``: extracted from ``tpep_pickup_datetime``.
5. ``tip_percentage``: ``tip_amount` / fare_amount * 100`` (handle division by zero).
6. ``speed_mph``: ``trip_distance` / (trip_duration_minutes / 60)`` (handle division by zero, filter out unreasonable speeds > 100 mph).

```

# 2.4
df_clean = df_clean.withColumn("trip_duration_minutes", (unix_timestamp("tpep_dropoff_datetime") - unix_timestamp("tpep_pickup_datetime")) / 60)
df_clean = df_clean.withColumn("hour_of_day", hour("tpep_pickup_datetime"))
df_clean = df_clean.withColumn("day_of_week", when(dayofweek("tpep_pickup_datetime") == 1, 7).otherwise(dayofweek("tpep_pickup_datetime") - 1))
df_clean = df_clean.withColumn("month", month("tpep_pickup_datetime"))
df_clean = df_clean.withColumn("tip_percentage", when(col("fare_amount") > 0, (col("tip_amount") / col("fare_amount")) * 100).otherwise(None))
df_clean = df_clean.withColumn("speed_mph", when(col("trip_duration_minutes") > 0, col("trip_distance") / (col("trip_duration_minutes") / 60)).otherwise(0))
df_clean = df_clean.filter((col("speed_mph").isNull()) | (col("speed_mph") <= 100))

```

```

df_clean.select(
    "trip_duration_minutes",
    "hour_of_day",
    "day_of_week",
    "month",
    "tip_percentage",
    "speed_mph"
).show(5)

```

trip_duration_minutes	hour_of_day	day_of_week	month	tip_percentage	speed_mph
17.7	0	2	10	8.152173913043478	10.16949152542373
13.083333333333334	0	2	10	26.76056338028169	10.089171974522294
9.1	0	2	10	27.40740740740741	17.802197802197803
10.85	0	2	10	14.084507042253522	17.142857142857146
4.666666666666667	0	2	10	33.888888888888886	12.471428571428572

only showing top 5 rows

2.5 Aggregations

Using the enriched DataFrame:

1. Average trip distance and fare amount per `hour\_of\_day`. What hours have the longest/most expensive trips?

Average trip distance and fare amount per hour of day:

```
hourly = df_clean.groupBy("hour_of_day").agg(
    avg("trip_distance").alias("avg_trip_distance"),
    avg("fare_amount").alias("avg_fare_amount")
).orderBy("hour_of_day")
hourly.show(24, truncate=False)
```

[Stage 145:=====>

hour_of_day	avg_trip_distance	avg_fare_amount
0	3.9940352209452854	20.30939805135777
1	3.451589763554973	18.065711786363174
2	3.093273722564794	16.579505236369506
3	3.36976697139736	17.58139394102555
4	4.964229166791558	23.816265099242834
5	6.338921588625854	28.514970320356465
6	4.942863689279742	23.142339296536758
7	3.743502452714594	19.53600729558587
8	3.224630307714529	18.576205212677593
9	3.1320892408323777	18.649960359949574
10	3.1565001066716376	19.039094384865432
11	3.130636535000449	19.304061133703566
12	3.221441404134355	19.73083647909722
13	3.412610298999755	20.460779123627834
14	3.56983635504181	21.25179492323656
15	3.5365677995660953	21.236029609032595
16	3.558670158640665	21.29922776964351
17	3.19673029175373	19.745882419992263
18	3.0193636938171844	18.44609488120487
19	3.2298187888769925	18.595324343158545
20	3.4365619170135546	18.969063553166333
21	3.486150854466687	19.044063138214394
22	3.650241817079383	19.657300458003647
23	4.0789334949131	20.96333551394519

Longest trips:

```
hourly.orderBy(col("avg_trip_distance").desc()).show(5)
```

[Stage 148:=====>

hour_of_day	avg_trip_distance	avg_fare_amount
5	6.338921588625854	28.514970320356465
4	4.964229166791558	23.816265099242834
6	4.942863689279742	23.142339296536758
23	4.0789334949131	20.96333551394519
0	3.9940352209452854	20.30939805135777

only showing top 5 rows

Most expensive trips:


```
hourly.orderBy(col("avg_fare_amount").desc()).show(5)
```

```
[Stage 151:=====>
+-----+-----+-----+
|hour_of_day|avg_trip_distance|  avg_fare_amount|
+-----+-----+-----+
|          5|6.338921588625854|28.514970320356465|
|          4|4.964229166791558|23.816265099242834|
|          6|4.942863689279742|23.142339296536758|
|         16|3.558670158640665| 21.29922776964351|
|         14| 3.56983635504181| 21.25179492323656|
+-----+-----+-----+
```

only showing top 5 rows

2. Total revenue per `month`. Is there a seasonal pattern?

```
df_clean.groupBy("month").agg(sum("total_amount").alias("total_revenue")).orderBy("month").show()
```

```
[Stage 166:=====> (8 + 4) / 12]
+-----+-----+
|month|    total_revenue|
+-----+-----+
|    1|7.461821098993786E7|
|    2|7.444384533994398E7|
|    3|8.601527200986908E7|
|    4|8.579874491987401E7|
|    5|9.370999945984167E7|
|    6|8.745999504986383E7|
|    7|7.899976238991883E7|
|    8|7.722124146991742E7|
|    9|9.131358298986937E7|
|   10|9.870875900983047E7|
|   11|9.077769155985183E7|
|   12|9.445349867983595E7|
+-----+-----+
```

3. Count of trips per `day\_of\_week`. Which days are busiest?

Trips per day of week

```
trips_per_day = df_clean.groupBy("day_of_week").agg(count("*").alias("trip_count")).orderBy("day_of_week")
trips_per_day.show()
```

```
[Stage 172:=====> (11 + 1) / 12]
+-----+-----+
|day_of_week|trip_count|
+-----+-----+
|          1|  4547041|
|          2|  5200650|
|          3|  5369091|
|          4|  5584259|
|          5|  5282992|
|          6|  5236548|
|          7|  4402983|
+-----+-----+
```

Busiest day:

```
trips_per_day.orderBy(col("trip_count").desc()).show()
```

```
[Stage 175:=====>
+-----+-----+
|day_of_week|trip_count|
+-----+-----+
|          4|  5584259|
|          3|  5369091|
|          5|  5282992|
|          6|  5236548|
|          2|  5200650|
|          1|  4547041|
|          7|  4402983|
+-----+-----+
```

4. Average `tip\_percentage` per `payment\_type`. Which payment types tip more?

```
df_clean.groupBy("payment_type").agg(avg("tip_percentage").alias("avg_tip_percentage")).orderBy(col("avg_tip_percentage").desc()).show()
```

```
[Stage 178:=====>      (11 + 1) / 12]
```

payment_type	avg_tip_percentage
1	25.097827415209125
4	0.06306863370604723
3	0.06155066243447759
2	0.001491349102521...

2.6 Write cleaned data

Write your cleaned and enriched DataFrame as a single Parquet file to `/app/data/nyc_taxi_2024_clean.parquet`. You will use this file in the remaining parts. Hint: use `.coalesce(1)` before `.write.parquet()` to produce a single file

```
# Overwrite to ensure rerunning the notebook does not fail
df_clean.coalesce(1).write.mode("overwrite").parquet("/app/data/nyc_taxi_2024_clean.parquet")
print(os.listdir("/app/data/nyc_taxi_2024_clean.parquet"))
```

```
['.part-00000-9e0b7ff2-a60e-430d-a309-744c21c109e3-c000.snappy.parquet.crc', '_SUCCESS.crc', 'part-00000-9e0b7ff2-a60e-430d-a309-744c21c109e3-c000.snappy.parquet', '_SUCCESS']
```

3. Spark SQL & Windows functions (35%)

Register your cleaned DataFrame as a temporary view and answer the following questions using Spark SQL (not the DataFrame API). This connects back to Session 07. `df_clean.createOrReplaceTempView("trips")`

3.1 SQL Basics

1. What are the top 10 busiest pickup locations (`PULocationID`) by number of trips?

```

top_locations = spark.sql("""
SELECT
    PULocationID,
    COUNT(*) AS trip_count
FROM trips
GROUP BY PULocationID
ORDER BY trip_count DESC
LIMIT 10
""")

top_locations.show()

```

[Stage 188:=====

PULocationID	trip_count
132	1856382
237	1766762
161	1737647
236	1557671
162	1309910
186	1269313
230	1250688
138	1245356
142	1182683
163	1063249

2. What is the average fare, tip, and total amount per payment type?

```

payment_stats = spark.sql("""
SELECT
    payment_type,
    AVG(fare_amount) AS avg_fare,
    AVG(tip_amount) AS avg_tip,
    AVG(total_amount) AS avg_total
FROM trips
GROUP BY payment_type
ORDER BY payment_type
""")

payment_stats.show()

```

[Stage 191:=====> (8 + 4) / 12]

payment_type	avg_fare	avg_tip	avg_total
1	19.746730828711478	4.35795764517939	29.753365123079842
2	19.636604169403547	2.381840884178569...	24.95892488719613
3	19.94209428067393	0.006390845307513097	25.267221676071134
4	22.256116792579085	0.01376143563621591	27.913546720416065

3. Use a CTE to find pickup locations where the average trip distance is more than twice the overall average trip distance.

```
distances = spark.sql("""
WITH overall_avg AS (
    SELECT AVG(trip_distance) AS overall_distance
    FROM trips
),

location_avg AS (
    SELECT
        PULocationID,
        AVG(trip_distance) AS avg_distance
    FROM trips
    GROUP BY PULocationID
)

SELECT
    l.PULocationID,
    l.avg_distance,
    o.overall_distance
FROM location_avg l
CROSS JOIN overall_avg o
WHERE l.avg_distance > 2 * o.overall_distance
ORDER BY l.avg_distance DESC
""")

distances.show()
```

[Stage 201:=====>

PULocationID	avg_distance	overall_distance
44	22.875	3.4374351914925456
156	18.413333333333334	3.4374351914925456
132	15.96060854931797	3.4374351914925456
86	15.776257253384912	3.4374351914925456
23	15.224499999999997	3.4374351914925456
2	14.90076923076923	3.4374351914925456
117	14.449293833964864	3.4374351914925456
115	13.928999999999998	3.4374351914925456
201	13.908612862547287	3.4374351914925456
118	13.837222222222222	3.4374351914925456
46	13.56572192513369	3.4374351914925456
219	13.381684287276707	3.4374351914925456
139	13.001226190476189	3.4374351914925456
10	12.940180454845086	3.4374351914925456
38	12.79779322853688	3.4374351914925456
259	12.706564569536424	3.4374351914925456
59	12.558823529411766	3.4374351914925456
203	12.502666666666667	3.4374351914925456
191	12.458740350877191	3.4374351914925456
184	12.136567164179104	3.4374351914925456

only showing top 20 rows

3.2 CASE Expressions

Write a query that classifies each trip into a `time\_of\_day` category:

- 6-11: 'morning'
- 12-17: 'afternoon'
- 18-22: 'evening'
- 23-5: 'night'

Then compute the average `trip\_distance`, `total\_amount`, and `tip\_percentage` per `time\_of\_day`. When do the most expensive trips happen?

```
# 3.2
time_of_day_trips = spark.sql("""
SELECT
    CASE
        WHEN hour_of_day BETWEEN 6 AND 11 THEN 'morning'
        WHEN hour_of_day BETWEEN 12 AND 17 THEN 'afternoon'
        WHEN hour_of_day BETWEEN 18 AND 22 THEN 'evening'
        ELSE 'night'
    END AS time_of_day,
    AVG(trip_distance) AS avg_trip_distance,
    AVG(total_amount) AS avg_total_amount,
    AVG(tip_percentage) AS avg_tip_percentage
FROM trips
GROUP BY
    CASE
        WHEN hour_of_day BETWEEN 6 AND 11 THEN 'morning'
        WHEN hour_of_day BETWEEN 12 AND 17 THEN 'afternoon'
        WHEN hour_of_day BETWEEN 18 AND 22 THEN 'evening'
        ELSE 'night'
    END
ORDER BY avg_total_amount DESC
""")

time_of_day_trips.show()
```

```
[Stage 206:=====> (9 + 3) / 12]
+-----+-----+-----+-----+
|time_of_day| avg_trip_distance|  avg_total_amount| avg_tip_percentage|
+-----+-----+-----+-----+
|  afternoon|3.4164588028984992| 30.00134694717678| 20.34933824213464|
|    night|3.9625570681088558|29.492232544607976|21.451774411315228|
|  evening|3.3407432265304293|28.717754586344608|22.551773768030156|
|  morning|3.3350354849338553|27.476839481322337|19.922032409207173|
+-----+-----+-----+-----+
```

The most expensive trips happen in the afternoon

3.3 Window Functions: Ranking

1. For each month, rank the top 5 pickup locations by total revenue using `ROW\_NUMBER`. Show month, location, revenue, and rank.

```
# 3.3
monthly_ranking = spark.sql("""
WITH monthly_revenue AS (
    SELECT
        month,
        PULocationID,
        SUM(total_amount) AS revenue
    FROM trips
    GROUP BY month, PULocationID
),

locations AS (
    SELECT
        month,
        PULocationID,
        revenue,
        ROW_NUMBER() OVER (
            PARTITION BY month
            ORDER BY revenue DESC
        ) AS revenue_rank
    FROM monthly_revenue
)

SELECT
    month,
    PULocationID,
    revenue,
    revenue_rank
FROM locations
WHERE revenue_rank <= 5
ORDER BY month, revenue_rank
""")

monthly_ranking.show(5, truncate=False)
```

[Stage 209:=====>

month	PULocationID	revenue	revenue_rank
1	132	1.107120876000486E7	1
1	138	5745056.440000303	2
1	161	3234330.899999876	3
1	237	2672667.109999881	4
1	230	2657710.9999998803	5

only showing top 5 rows

2. Use `NTILE(4)` to divide pickup locations into revenue quartiles per month. How many locations are in the top quartile?

```

revenue_quartiles = spark.sql("""
WITH monthly_revenue AS (
    SELECT
        month,
        PULocationID,
        SUM(total_amount) AS revenue
    FROM trips
    GROUP BY month, PULocationID
),

quartiles AS (
    SELECT
        month,
        PULocationID,
        revenue,
        NTILE(4) OVER (
            PARTITION BY month
            ORDER BY revenue DESC
        ) AS revenue_quartile
    FROM monthly_revenue
)

SELECT
    month,
    COUNT(*) AS top_quartile_locations
FROM quartiles
WHERE revenue_quartile = 1
GROUP BY month
ORDER BY month
""")

revenue_quartiles.show()

```

[Stage 215:=====>

month	top_quartile_locations
1	64
2	63
3	63
4	63
5	63
6	63
7	64
8	63
9	63
10	65
11	64
12	63

3.4 Window Functions: Running Totals and Moving Averages

1. Compute daily total revenue (sum of `total\_amount` per day).
2. Add a running total of revenue that accumulates over the year.
3. Add a 7-day moving average of daily revenue.
4. Add a column showing the day-over-day revenue change using `LAG`.

What was the biggest single-day drop in revenue?

```
# 3.4
daily_revenue = spark.sql("""
WITH daily_stats AS (
    SELECT
        DATE(tpcp_pickup_datetime) AS trip_date,
        SUM(total_amount) AS daily_revenue
    FROM trips
    GROUP BY DATE(tpcp_pickup_datetime)
)

SELECT
    trip_date,
    daily_revenue,
    SUM(daily_revenue) OVER (
        ORDER BY trip_date
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS running_total_revenue,
    AVG(daily_revenue) OVER (
        ORDER BY trip_date
        ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
    ) AS moving_avg_7d,
    daily_revenue - LAG(daily_revenue) OVER (
        ORDER BY trip_date
    ) AS revenue_change
FROM daily_stats
ORDER BY trip_date
""")

daily_revenue.show(10, truncate=False)
```

trip_date	daily_revenue	running_total_revenue	moving_avg_7d	revenue_change
2024-01-01	2102786.3999998653	2102786.3999998653	2102786.3999998653	NULL
2024-01-02	2179962.0899998774	4282748.489999743	2141374.2449998716	77175.69000001205
2024-01-03	2265092.2499999288	6547840.739999672	2182613.5799998906	85130.16000005137
2024-01-04	2687180.779999942	9235021.519999614	2308755.3799999035	422088.5300000133
2024-01-05	2600641.5899999337	1.1835663109999549E7	2367132.6219999096	-86539.19000000833
2024-01-06	2279012.1899999455	1.4114675299999494E7	2352445.883333249	-321629.39999998827
2024-01-07	1799281.6599998763	1.5913956959999371E7	2273422.422857053	-479730.53000000692
2024-01-08	2123706.3899999345	1.8037663349999305E7	2276410.9928570623	324424.73000000582
2024-01-09	2161180.8499999866	2.019884419999929E7	2273727.9585713637	37474.460000005212
2024-01-10	2426964.2399999807	2.262580843999927E7	2296852.528571371	265783.3899999941

only showing top 10 rows

Biggest selling day

```
# Best selling day revenue
best_selling = spark.sql("""
WITH daily_stats AS (
    SELECT
        DATE(tpcp_pickup_datetime) AS trip_date,
        SUM(total_amount) AS daily_revenue
    FROM trips
    GROUP BY DATE(tpcp_pickup_datetime)
),

revenue_changes AS (
    SELECT
        trip_date,
        daily_revenue,
        daily_revenue - LAG(daily_revenue) OVER (
            ORDER BY trip_date
        ) AS revenue_change
    FROM daily_stats
)

SELECT *
FROM revenue_changes
ORDER BY revenue_change ASC
LIMIT 1
""")

best_selling.show()
```

```
+-----+-----+-----+
| trip_date|   daily_revenue|revenue_change|
+-----+-----+-----+
|2024-01-01|2102786.3999998653|          NULL|
+-----+-----+-----+
```

3.5 Window Functions: Comparisons

1. For each trip, show the trip's `total\_amount` alongside the average total_amount for that hour of day. Use a window function (not a self-join or subquery). Which trips are most above their hourly average?

```
hourly_comparison = spark.sql("""
SELECT
    tpep_pickup_datetime,
    hour_of_day,
    total_amount,
    AVG(total_amount) OVER (
        PARTITION BY hour_of_day
    ) AS hourly_avg_total,
    total_amount - AVG(total_amount) OVER (
        PARTITION BY hour_of_day
    ) AS difference_hour
FROM trips
ORDER BY difference_hour DESC
""")

hourly_comparison.show()
```

```
[Stage 265:=====> (6 + 1) / 7]
+-----+-----+-----+-----+-----+
|tpep_pickup_datetime|hour_of_day|total_amount| hourly_avg_total| difference_hour|
+-----+-----+-----+-----+-----+
| 2024-12-04 14:24:18|      14|    3037.1| 30.10276034924424|3006.9972396507555|
| 2024-07-22 20:41:53|      20|    2265.45|28.448740387291007| 2237.001259612709|
| 2024-01-14 10:08:11|      10|    2225.3|27.318412750134847|2197.9815872498652|
| 2024-04-06 16:33:28|      16|    1737.18| 32.07531533721916| 1705.104684662781|
| 2024-11-06 09:05:55|       9|    1477.68|26.796420912531172| 1450.883579087469|
| 2024-06-30 18:20:04|      18|    1315.97| 28.68883199424098|1287.2811680057591|
| 2024-12-16 17:06:06|      17|    1218.99|30.211947044066807|1188.7780529559332|
| 2024-08-02 18:01:02|      18|    1204.41| 28.68883199424098|1175.7211680057592|
| 2024-08-06 23:14:08|      23|    1199.67|30.748023929528355|1168.9219760704718|
| 2024-07-18 00:53:39|       0|    1150.67| 29.68945619431073|1120.9805438056894|
| 2024-07-11 22:33:59|      22|    1111.54| 29.14116946132411|1082.3988305386758|
| 2024-04-27 23:33:40|      23|    1078.87|30.748023929528355|1048.1219760704716|
| 2024-11-04 10:23:19|      10|    1067.58|27.318412750134847| 1040.261587249865|
| 2024-03-03 22:38:02|      22|    1021.99| 29.14116946132411| 992.8488305386759|
| 2024-04-27 23:36:57|      23|    1018.97|30.748023929528355| 988.2219760704717|
| 2024-05-14 20:15:22|      20|    1001.5|28.448740387291007| 973.051259612709|
| 2024-02-18 14:13:36|      14|    1000.0| 30.10276034924424| 969.8972396507558|
| 2024-04-29 06:09:11|       6|     998.5|31.863105720054037| 966.636894279946|
| 2024-07-29 23:54:51|      23|     971.0|30.748023929528355| 940.2519760704716|
| 2024-07-15 14:43:57|      14|     965.18| 30.10276034924424| 935.0772396507557|
+-----+-----+-----+-----+-----+
only showing top 20 rows
```

2. Use `FIRST\_VALUE` and `LAST\_VALUE` (with the correct frame!) to show, for each month, the date with the highest and lowest daily revenue.

```

highest_lowest_revenue = spark.sql("""
WITH daily_revenue AS (
    SELECT
        month,
        DATE(tpep_pickup_datetime) AS trip_date,
        SUM(total_amount) AS daily_revenue
    FROM trips
    GROUP BY month, DATE(tpep_pickup_datetime)
)

SELECT DISTINCT
    month,
    FIRST_VALUE(trip_date) OVER (
        PARTITION BY month
        ORDER BY daily_revenue DESC
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    ) AS highest_revenue_day,
    FIRST_VALUE(daily_revenue) OVER (
        PARTITION BY month
        ORDER BY daily_revenue DESC
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    ) AS highest_revenue,
    LAST_VALUE(trip_date) OVER (
        PARTITION BY month
        ORDER BY daily_revenue DESC
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    ) AS lowest_revenue_day,
    LAST_VALUE(daily_revenue) OVER (
        PARTITION BY month
        ORDER BY daily_revenue DESC
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    ) AS lowest_revenue
FROM daily_revenue
ORDER BY month

```

```

[Stage 266:=====> (9 + 3) / 12]
+-----+-----+-----+-----+-----+
|month|highest_revenue_day|highest_revenue|lowest_revenue_day|lowest_revenue|
+-----+-----+-----+-----+-----+
| 1|2024-01-25|2827199.5599999046|2024-01-07|1799281.65999998763|
| 2|2024-02-29|3175067.0899998588|2024-02-13|1636503.4299999436|
| 3|2024-03-14|3355273.759999981|2024-03-31|2303892.26999998557|
| 4|2024-04-18|3358428.489999808|2024-04-22|2420538.5399998706|
| 5|2024-05-16|3645496.9999997504|2024-05-27|1895569.03999998894|
| 6|2024-06-06|3584550.7399997856|2024-06-30|2208535.3999998807|
| 7|2024-07-18|3162952.4099998423|2024-07-04|1489733.1299999114|
| 8|2024-08-01|2959176.5499998326|2024-08-18|2103687.6499998774|
| 9|2024-09-26|3599853.9599997164|2024-09-01|2253963.1299998746|
|10|2024-10-17|3729636.5399997947|2024-10-14|2669777.6299998933|
|11|2024-11-21|3859686.049999713|2024-11-28|1637755.1099999251|
|12|2024-12-12|4368145.609999804|2024-12-25|1253865.019999973|
+-----+-----+-----+-----+-----+

```

3.6 ROLLUP

Use `ROLLUP` to compute total revenue with subtotals by `month` and `payment\_type`. Identify the NULL rows and explain what they represent.

```
total_revenue = spark.sql("""
SELECT
    month,
    payment_type,
    SUM(total_amount) AS total_revenue
FROM trips
GROUP BY ROLLUP(month, payment_type)
ORDER BY month, payment_type
""")

total_revenue.show()
```

[Stage 272:=====;

month	payment_type	total_revenue
NULL	NULL	1.0335206038685542E9
1	NULL	7.461821098993786E7
1	1	6.3853269359997004E7
1	2	9979769.850000516
1	3	217962.16000000213
1	4	567209.620000004
2	NULL	7.444384533994398E7
2	1	6.434241821999472E7
2	2	9293911.060000887
2	3	221516.05000000112
2	4	586000.0099999999
3	NULL	8.601527200986908E7
3	1	7.40164440499379E7
3	2	1.0960267689999329E7
3	3	282517.2400000034
3	4	756043.0299999764
4	NULL	8.579874491987401E7
4	1	7.367574222994539E7
4	2	1.1093835020000206E7
4	3	274852.4400000013

The null values represent subtotal and total aggregation levels.

- When payment_type is null, the row represents the total revenue for that month across all payment types.
- If both month and payment_type are null, that row represents the grand total revenue for the entire dataset.

Part 4: Performance Analysis (10%)

4.1 Query plans

Pick your daily revenue query with the running total and 7-day moving average from Part 3.4 and call `explain(True)` on it. In your report, describe:

1. What does the physical plan show? How many stages can you identify?

```
daily_revenue.explain(True)

== Physical Plan ==
AdaptiveSparkPlan isFinalPlan=false
+- Project [trip_date#2610, daily_revenue#2611, running_total_revenue#2607, moving_avg_7d#2608, (daily_revenue#2611 - _we2#2615) AS revenue_change#2609]
   +- Window [sum(daily_revenue#2611) window[specdefinition(trip_date#2610 ASC NULLS FIRST, specifiedwindowframe(RowFrame, unboundedpreceding$(), currentrow$())) AS running_total_revenue#2607, avg(daily_revenue#2611) window[specdefinition(trip_date#2610 ASC NULLS FIRST, specifiedwindowframe(RowFrame, -6, currentrow$())) AS moving_avg_7d#2608, lag(daily_revenue#2611, -1, null) window[specdefinition(trip_date#2610 ASC NULLS FIRST, specifiedwindowframe(RowFrame, -1, -1)) AS _we2#2615], [trip_date#2610 ASC NULLS FIRST]]
      +- Sort [trip_date#2610 ASC NULLS FIRST], false, 0
         +- Exchange SinglePartition, ENSURE_REQUIREMENTS, [plan_id=5126]
            +- HashAggregate(keys=[_groupingexpression#3230], functions=[sum(total_amount#35)], output=[trip_date#2610, daily_revenue#2611])
               +- Exchange hashpartitioning(_groupingexpression#3230, 8), ENSURE_REQUIREMENTS, [plan_id=5123]
                  +- HashAggregate(keys=[_groupingexpression#3230], functions=[partial_sum(total_amount#35)], output=[_groupingexpression#3230, sum#2623])
                     +- Project [total_amount#35, cast(tpep_pickup_datetime#20 as date) AS _groupingexpression#3230]
                        +- Filter (((((((isnotnull(trip_distance#23) AND isnotnull(total_amount#35)) AND isnotnull(tpep_pickup_datetime#20)) AND (trip_distance#23 > 0.0)) AND (total_amount#35 > 0.0)) AND isnotnull(passenger_count#22L) AND (passenger_count#22L > 0.0)) AND (trip_distance#23 <= 20.0)) AND (total_amount#35 <= 5000.0)) AND (tpep_pickup_datetime#20 >= 2024-01-01 00:00:00) AND (tpep_pickup_datetime#20 < 2025-01-01 00:00:00)) AND (CASE WHEN (((cast((unix_timestamp(tpep_dropoff_datetime#21, yyyy-MM-dd HH:mm:ss, Some(Etc/UTC), true) - unix_timestamp(tpep_pickup_datetime#20, yyyy-MM-dd HH:mm:ss, Some(Etc/UTC), true)) as double) / 60.0) > 0.0) THEN isnull((trip_distance#23 / ((cast((unix_timestamp(tpep_dropoff_datetime#21, yyyy-MM-dd HH:mm:ss, Some(Etc/UTC), true) - unix_timestamp(tpep_pickup_datetime#20, yyyy-MM-dd HH:mm:ss, Some(Etc/UTC), true)) as double) / 60.0) / 60.0))) ELSE true END OR CASE WHEN (((cast((unix_timestamp(tpep_dropoff_datetime#21, yyyy-MM-dd HH:mm:ss, Some(Etc/UTC), true) - unix_timestamp(tpep_pickup_datetime#20, yyyy-MM-dd HH:mm:ss, Some(Etc/UTC), true)) as double) / 60.0) > 0.0) THEN ((trip_distance#23 / ((cast((unix_timestamp(tpep_dropoff_datetime#21, yyyy-MM-dd HH:mm:ss, Some(Etc/UTC), true) - unix_timestamp(tpep_pickup_datetime#20, yyyy-MM-dd HH:mm:ss, Some(Etc/UTC), true)) as double) / 60.0) / 60.0)) <= 100.0) ELSE false END)))
                           +- FileScan parquet [tpep_pickup_datetime#20,tpep_dropoff_datetime#21,passenger_count#22L,trip_distance#23,total_amount#35] Batched: true, DataFilters: [isnotnull(trip_distance#23), isnotnull(total_amount#35), isnotnull(tpep_pickup_datetime#20), (tr..., Format: Parquet, Location: InMemoryFileIndex(12 paths)[file:/app/data/yellow_tripdata_2024-01.parquet, file:/app/data/yellow..., PartitionFilters: [], PushedFilters: [IsNotNull(trip_distance), IsNotNull(total_amount), IsNotNull(tpep_pickup_datetime), GreaterThan(..., ReadSchema: struct<tpep_pickup_datetime:timestamp_ntz,tpep_dropoff_datetime:timestamp_ntz,passenger_count:big...
```

The physical plan shows that Spark reads the Parquet files, applies the cleaning filters, projects only the needed columns, aggregates revenue by date, sorts the daily results, and then applies the window functions for running total, 7-day moving average, and day-over-day revenue change.

Three stages were identified:

- Parquet scan, filtering, projection, and partial aggregation.
- Shuffle and final aggregation by trip_date
- Single-partition sorting and window calculation.

2. Can you identify where shuffles happen? How many are there and why?

There are 2 shuffles:

- Exchange hashpartitioning(_groupingexpression#3230, 8):
Since Spark groups data by pickup date to compute daily revenue, rows with the same date may be spread across different partitions, so Spark must redistribute the data by date.
- Exchange SinglePartition:
Since the window functions use ORDER BY trip_date without a PARTITION BY. Spark

moves the daily aggregated data into a single partition so it can compute the running total, moving average, and LAG() in chronological order.

3. Is Spark reading all columns from the Parquet file, or only some? How can you tell from the plan?

Spark is reading only some columns, not all columns. This is shown by:

+ FileScan parquet [tpep_pickup_datetime#20, tpep_dropoff_datetime#21, passenger_count#22L, trip_distance#23, total_amount#35]

The plan only reads the needed columns.

4.2 Caching

Choose an aggregation query that you run multiple times (e.g., the daily revenue from 3.4).

Measure the execution time:

1. Without caching (run 3 times, report average).

```
query = """
WITH daily_stats AS (
  SELECT
    DATE(tpep_pickup_datetime) AS trip_date,
    SUM(total_amount) AS daily_revenue
  FROM trips
  GROUP BY DATE(tpep_pickup_datetime)
)

SELECT
  trip_date,
  daily_revenue,
  SUM(daily_revenue) OVER (
    ORDER BY trip_date
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
  ) AS running_total_revenue,
  AVG(daily_revenue) OVER (
    ORDER BY trip_date
    ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
  ) AS moving_avg_7d,
  daily_revenue - LAG(daily_revenue) OVER (
    ORDER BY trip_date
  ) AS revenue_change
FROM daily_stats
ORDER BY trip_date
"""

times_no_cache = []

for i in range(3):
    start = time.time()
    spark.sql(query).count()
    end = time.time()
    times_no_cache.append(end - start)

avg_no_cache = builtins.sum(times_no_cache) / len(times_no_cache)
print("Average time without cache:", avg_no_cache)
```

```
[Stage 57:=====>
Average time without cache: 2.396759351094564
```

2. With `.cache()` on the base DataFrame (run 3 times, report average).


```
# Cache DataFrame
df_clean.cache()
df_clean.count()

times_cache = []
for i in range(3):
    start = time.time()
    spark.sql(query).count()
    end = time.time()
    times_cache.append(end - start)
avg_cache = builtins.sum(times_cache) / len(times_cache)

print("Average time with cache:", avg_cache)
```

```
26/05/09 14:56:32 WARN CacheManager: Asked to cache already cached data.
26/05/09 14:56:32 WARN MemoryStore: Not enough space to cache rdd_128_2 in memory! (computed 20.6 MiB so far)
26/05/09 14:56:32 WARN MemoryStore: Not enough space to cache rdd_128_5 in memory! (computed 40.1 MiB so far)
26/05/09 14:56:32 WARN MemoryStore: Not enough space to cache rdd_128_9 in memory! (computed 20.6 MiB so far)
26/05/09 14:56:32 WARN MemoryStore: Not enough space to cache rdd_128_8 in memory! (computed 20.7 MiB so far)
26/05/09 14:56:32 WARN MemoryStore: Not enough space to cache rdd_128_4 in memory! (computed 39.9 MiB so far)
26/05/09 14:56:32 WARN MemoryStore: Not enough space to cache rdd_128_10 in memory! (computed 20.6 MiB so far)
26/05/09 14:56:33 WARN MemoryStore: Not enough space to cache rdd_128_5 in memory! (computed 20.6 MiB so far)
26/05/09 14:56:34 WARN MemoryStore: Not enough space to cache rdd_128_2 in memory! (computed 39.9 MiB so far)
26/05/09 14:56:34 WARN MemoryStore: Not enough space to cache rdd_128_4 in memory! (computed 39.9 MiB so far)
26/05/09 14:56:36 WARN MemoryStore: Not enough space to cache rdd_128_9 in memory! (computed 79.0 MiB so far)
26/05/09 14:56:36 WARN MemoryStore: Not enough space to cache rdd_128_11 in memory! (computed 40.1 MiB so far)
26/05/09 14:56:38 WARN MemoryStore: Not enough space to cache rdd_128_4 in memory! (computed 20.6 MiB so far)
26/05/09 14:56:38 WARN MemoryStore: Not enough space to cache rdd_128_3 in memory! (computed 40.1 MiB so far)
26/05/09 14:56:38 WARN MemoryStore: Not enough space to cache rdd_128_2 in memory! (computed 20.6 MiB so far)
26/05/09 14:56:38 WARN MemoryStore: Not enough space to cache rdd_128_11 in memory! (computed 40.1 MiB so far)
26/05/09 14:56:39 WARN MemoryStore: Not enough space to cache rdd_128_9 in memory! (computed 215.4 MiB so far)
26/05/09 14:56:40 WARN MemoryStore: Not enough space to cache rdd_128_4 in memory! (computed 39.9 MiB so far)
26/05/09 14:56:41 WARN MemoryStore: Not enough space to cache rdd_128_9 in memory! (computed 79.0 MiB so far)
26/05/09 14:56:41 WARN MemoryStore: Not enough space to cache rdd_128_11 in memory! (computed 79.1 MiB so far)
26/05/09 14:56:41 WARN MemoryStore: Not enough space to cache rdd_128_10 in memory! (computed 78.9 MiB so far)
26/05/09 14:56:43 WARN MemoryStore: Not enough space to cache rdd_128_2 in memory! (computed 329.7 MiB so far)
26/05/09 14:56:43 WARN MemoryStore: Not enough space to cache rdd_128_0 in memory! (computed 214.1 MiB so far)
26/05/09 14:56:43 WARN MemoryStore: Not enough space to cache rdd_128_3 in memory! (computed 214.6 MiB so far)
[Stage 82:=====> (10 + 2) / 12]
Average time with cache: 3.3796013991038003
```

Is there a difference? Why or why not? (Hint: the answer may surprise you; explain what you observe, even if caching doesn't help. If that's the case, what could be the cause.)

The average execution time without caching was approximately 2.4 seconds, while the version with cache took around 3.38 seconds. This happened because the dataset was too large to fit entirely in memory, causing Spark to spill cached partitions to disk.

In our case, caching introduced additional overhead from serialization, memory management, and disk persistence, outweighing the performance benefits. Also, the query already benefits from Parquet optimizations, so Spark was already reading the data efficiently from disk.

4.3 Partitioned reads

In Part 2.6 you wrote a single Parquet file. Now write the same data partitioned by month. Then measure the time to read and filter for a single month:

1. Reading from the single file + filtering in memory.
2. Reading from the partitioned dataset (partition pruning).

Report the difference. Check `explain()` to confirm partition pruning is happening.


```
# 4.3
```

```
# Write partitioned dataset
```

```
df_clean.write.mode("overwrite").partitionBy("month").parquet("/app/data/nyc_taxi_partitioned")
```

```
26/05/09 15:55:09 WARN MemoryStore: Not enough space to cache rdd_128_4 in memory! (computed 136.7 MiB so far)
26/05/09 15:55:09 WARN MemoryStore: Not enough space to cache rdd_128_2 in memory! (computed 136.3 MiB so far)
26/05/09 15:55:10 WARN MemoryStore: Not enough space to cache rdd_128_7 in memory! (computed 214.4 MiB so far)
26/05/09 15:55:10 WARN MemoryStore: Not enough space to cache rdd_128_3 in memory! (computed 214.6 MiB so far)
26/05/09 15:55:10 WARN MemoryStore: Not enough space to cache rdd_128_0 in memory! (computed 214.1 MiB so far)
26/05/09 15:55:59 WARN MemoryStore: Not enough space to cache rdd_128_10 in memory! (computed 137.1 MiB so far)
```

```
start = time.time()
single_file_df = spark.read.parquet("/app/data/nyc_taxi_2024_clean.parquet")
single_file_df.filter(col("month") == 1).count()
end = time.time()
print("Single file read time:", end - start)
```

```
Single file read time: 0.6354281902313232
```

```
start = time.time()
partitioned_df = spark.read.parquet("/app/data/nyc_taxi_partitioned")
partitioned_df.filter(col("month") == 1).count()
end = time.time()
print("Partitioned read time:", end - start)
```

```
Partitioned read time: 0.8308186531066895
```

The partitioned dataset was slightly slower than the single-file dataset. The single-file took 0.64 seconds, 0.2 seconds faster than the partitioned dataset.

Even though partition pruning was applied, the dataset is relatively small and stored locally, so it doesn't have as much benefit. Partitioning is more useful for larger datasets or distributed storage systems where avoiding full scans saves I/O time.

```
partitioned_df.filter(col("month") == 1).explain(True)
```

```
== Parsed Logical Plan ==
'Filter' <=> ('month', 1)
+- Relation [VendorID#4942,tpep_pickup_datetime#4943,tpep_dropoff_datetime#4944,passenger_count#4945L,trip_distance#4946,RatecodeID#4947L,store_and_fwd_flag#4948,PULocationID#4949,DOLocationID#4950,payment_type#4951L,fare_amount#4952,extra#4953,mta_tax#4954,tip_amount#4955,tolls_amount#4956,improvement_surcharge#4957,total_amount#4958,congestion_surcharge#4959,Airport_fee#4960,trip_duration_minutes#4961,hour_of_day#4962,day_of_week#4963,tip_percentage#4964,speed_mph#4965,month#4966] parquet

== Analyzed Logical Plan ==
VendorID: int, tpep_pickup_datetime: timestamp_ntz, tpep_dropoff_datetime: timestamp_ntz, passenger_count: bigint, trip_distance: double, RatecodeID: bigint, store_and_fwd_flag: string, PULocationID: int, DOLocationID: int, payment_type: bigint, fare_amount: double, extra: double, mta_tax: double, tip_amount: double, tolls_amount: double, improvement_surcharge: double, total_amount: double, congestion_surcharge: double, Airport_fee: double, trip_duration_minutes: double, hour_of_day: int, day_of_week: int, tip_percentage: double, speed_mph: double, month: int
Filter (month#4966 = 1)
+- Relation [VendorID#4942,tpep_pickup_datetime#4943,tpep_dropoff_datetime#4944,passenger_count#4945L,trip_distance#4946,RatecodeID#4947L,store_and_fwd_flag#4948,PULocationID#4949,DOLocationID#4950,payment_type#4951L,fare_amount#4952,extra#4953,mta_tax#4954,tip_amount#4955,tolls_amount#4956,improvement_surcharge#4957,total_amount#4958,congestion_surcharge#4959,Airport_fee#4960,trip_duration_minutes#4961,hour_of_day#4962,day_of_week#4963,tip_percentage#4964,speed_mph#4965,month#4966] parquet

== Optimized Logical Plan ==
Filter (isnotnull(month#4966) AND (month#4966 = 1))
+- Relation [VendorID#4942,tpep_pickup_datetime#4943,tpep_dropoff_datetime#4944,passenger_count#4945L,trip_distance#4946,RatecodeID#4947L,store_and_fwd_flag#4948,PULocationID#4949,DOLocationID#4950,payment_type#4951L,fare_amount#4952,extra#4953,mta_tax#4954,tip_amount#4955,tolls_amount#4956,improvement_surcharge#4957,total_amount#4958,congestion_surcharge#4959,Airport_fee#4960,trip_duration_minutes#4961,hour_of_day#4962,day_of_week#4963,tip_percentage#4964,speed_mph#4965,month#4966] parquet

== Physical Plan ==
*(1) ColumnarToRow
+- FileScan parquet [VendorID#4942,tpep_pickup_datetime#4943,tpep_dropoff_datetime#4944,passenger_count#4945L,trip_distance#4946,RatecodeID#4947L,store_and_fwd_flag#4948,PULocationID#4949,DOLocationID#4950,payment_type#4951L,fare_amount#4952,extra#4953,mta_tax#4954,tip_amount#4955,tolls_amount#4956,improvement_surcharge#4957,total_amount#4958,congestion_surcharge#4959,Airport_fee#4960,trip_duration_minutes#4961,hour_of_day#4962,day_of_week#4963,tip_percentage#4964,speed_mph#4965,month#4966] Batched: true, DataFilters: [], Format: Parquet, Location: InMemoryFileIndex(1 paths)[file:/app/data/nyc_taxi_partitioned], PartitionFilters: [isnotnull(month#4966), (month#4966 = 1)], PushedFilters: [], ReadSchema: struct<VendorID:int,tpep_pickup_datetime:timestamp_ntz,tpep_dropoff_datetime:timestamp_ntz,passen...
```

The explain(True) output confirms that partition pruning is happening. In the physical plan we see:

PartitionFilters: [isnotnull(month#4966), (month#4966 = 1)]

Which indicates that Spark is only reading the partition corresponding to the first month, instead of scanning all monthly partitions.

Part 5: Final summary Report (15%)

Write a short document (PDF) that answers the following questions. Support your answers with data and measurements from your notebook.

1. RDDs vs DataFrames: Based on Part 1 and Part 2, what are the concrete advantages of DataFrames over RDDs for structured data? Give specific examples from your code.

Overall, DataFrames were more practical and readable than RDDs. In the first part of this practical, we were required to do manual transformations like map, filter and reduce. For example calculating average trip distance required us to manually create pairs and reduce them

```
trip_distance = rdd_jan.map(lambda row: (row.trip_distance, 1)).reduce(lambda a, b: (a[0] + b[0], a[1] + b[1]))
avg_trip_distance = trip_distance[0] / trip_distance[1]
print("Average trip distance:", avg_trip_distance)
```

```
[Stage 6:=====> (6 + 2) / 8]
Average trip distance: 3.6521691789583146
```

On the other hand, DataFrames made it easier thanks to its functions like filter, groupBy, agg and letting the use of the column names directly. It also allowed Spark to optimize execution using schema information, column pruning and filter pushdown. For example, in 4.1 we show how Spark only uses the columns needed for the query:

```
# 4.1
daily_revenue.explain(True)

time#20, yyyy-MM-dd HH:mm:ss, Some(Etc/UTC), true)) as double) / 60.0) > 0.0) THEN ((trip_distance#23 / ((cast((unix_timestamp(tpep_dropoff_datetime#
21, yyyy-MM-dd HH:mm:ss, Some(Etc/UTC), true) - unix_timestamp(tpep_pickup_datetime#20, yyyy-MM-dd HH:mm:ss, Some(Etc/UTC), true)) as double) / 60.0)
/ 60.0)) <= 100.0) ELSE false END))
+- FileScan parquet [tpep_pickup_datetime#20,tpep_dropoff_datetime#21,passenger_count#22L,trip_distance#23,total_amount#35]
Batched: true, DataFilters: [isnotnull(trip_distance#23), isnotnull(total_amount#35), isnotnull(tpep_pickup_datetime#20), (tr..., Format: Parquet, Loc
ation: InMemoryFileIndex(12 paths)[file:/app/data/yellow_tripdata_2024-01.parquet, file:/app/data/yellow..., PartitionFilters: [], PushedFilters: [IsN
otNull(trip_distance), IsNotNull(total_amount), IsNotNull(tpep_pickup_datetime), GreaterThan(..., ReadSchema: struct<tpep_pickup_datetime:timestamp_nt
z,tpep_dropoff_datetime:timestamp_ntz,passenger_count:big...
```

2. SQL vs DataFrame API: You used the DataFrame API in Part 2 and SQL in Part 3. For the queries you wrote, which approach was more readable? When would you choose one over the other?

Both had their uses. DataFrame API was more readable for cleaning and feature engineering thanks to its ability to easily chain operations like filtering invalid trips, creating new columns, and computing aggregations.

And SQL was more readable for analytical queries, mainly in Part 3. Queries with CTE, CASE, ROLLUP, and window functions were much more easy to understand in SQL form. So I would choose DataFrames for ETL style transformations and SQL for reporting, aggregations, and analytical queries.

3. Data quality: What data quality issues did you find in the taxi dataset? What percentage of data did you remove, and what was the most common issue?

Several data quality issues were found in the taxi dataset, such as trips with zero or negative distance, zero or negative total amount, null or zero passenger counts, etc.

After cleaning, I removed 5537218 rows, which represented 13.45% of the dataset. And the most common issue was invalid passenger count, 4200008 invalid rows were removed.

```
Initial rows: 41169720
Invalid trips removed: 1336887
Invalid passenger count removed: 4200008
Outliers removed: 268
Time anomalies removed: 55
Final rows: 35632502
Total removed: 5537218
Total removed percentage: 13.45%
```

4. Window functions: Choose one of your window function queries from Part 3 and explain what it does step by step. What would the equivalent be without window functions (using subqueries or self-joins)?

```
hourly_comparison = spark.sql("""
SELECT
    tpep_pickup_datetime,
    hour_of_day,
    total_amount,
    AVG(total_amount) OVER (
        PARTITION BY hour_of_day
    ) AS hourly_avg_total,
    total_amount - AVG(total_amount) OVER (
        PARTITION BY hour_of_day
    ) AS difference_hour
FROM trips
ORDER BY difference_hour DESC
""")

hourly_comparison.show()
```

This window function query compares each trip's total_amount with the average total amount for the same hour of day.

First, the query keeps each individual trip row's data like the pickup time, hour of day, and total amount. Then, using `AVG(total_amount) OVER (PARTITION BY hour_of_day)`, we calculate the average trip total for each hour group. Finally, the query computes `difference_hour` by subtracting the hourly average from each trip's total amount.

Without window functions, this would require a subquery that first computes the average total_amount per hour_of_day, followed by a join back to the original trips table on hour_of_day.

With the window function approach, it makes this query much simpler and much more readable since it avoids additional joins while keeping all original rows.

5. Performance: Based on Part 4, what did you learn about how Spark executes queries? What optimization had the biggest impact? Do you think this will always be the case?

I learned that Spark transforms queries into physical execution plans composed of stages, shuffles, scans, aggregations, sorting, and window operations. With `explain(True)` I saw how Spark executes the query internally.

The most important optimization was column pruning and predicate pushdown. The physical plan showed that Spark only read the necessary columns instead of scanning the full dataset, reducing unnecessary I/O, while the `PartitionFilters` section showed the partition pruning.

I don't think this will always be the most impactful optimization. On larger datasets or distributed environments, caching or partitioning may provide greater performance improvements depending on the workload and storage system.

6. One thing you learned: What was the most surprising or interesting finding from this assignment?

The most surprising finding was that caching and partitioning did not improve performance. In the tests, the cached query was slower than the non-cached version, and the same happened with the partitioned dataset. This showed me that Spark optimizations depend heavily on other factors like size, memory availability, storage system, and query type, rather than always guaranteeing faster execution.